

Estructuras de Datos y Algoritmos

Tema 5: Recursión

Dr. Jorge Armando Ramírez Díaz

Universidad Nacional Autónoma de México
ENES Unidad Juriquilla
Licenciatura en Matemáticas para el Desarrollo

Intuición

Un algoritmo recursivo resuelve un problema llamándose a sí mismo sobre una o más instancias más pequeñas del mismo problema.

La idea central no es repetir indefinidamente, sino reducir el problema hasta llegar a un caso que podamos resolver directamente.

Esquema básico

problema grande $\longrightarrow$ problemas más pequeños $\longrightarrow$ caso base

Estructura de un algoritmo recursivo

Todo algoritmo recursivo correcto debe dejar claras tres partes:

- **Caso base:** instancia que se resuelve directamente, sin hacer una llamada recursiva.
- **Caso recursivo:** reducción del problema original a una o más instancias del mismo tipo.
- **Progreso:** cada llamada recursiva debe acercarse al caso base.

Si falta el caso base, o si las llamadas recursivas no progresan, el algoritmo puede no terminar.

Recursión directa e indirecta

Podemos distinguir dos formas de recursividad:

Recursión directa

Una función se llama a sí misma.

$$f \longrightarrow f$$

Recursión indirecta

Una función llama a otra función que, directa o indirectamente, vuelve a llamar a la primera.

$$f \longrightarrow g \longrightarrow f$$

Ejemplo: factorial

El factorial de un entero $n \geq 0$ se define por:

$$n! = \begin{cases} 1, & n = 0, \\ n \cdot (n-1)!, & n > 0. \end{cases}$$

Esta definición ya es recursiva:

- El caso base es $0! = 1$.
- El caso recursivo expresa $n!$ usando $(n-1)!$.
- El progreso ocurre porque $n-1 < n$.

Ejemplo: factorial en Python

```
def factorial(n):  
    if n == 0:  
        return 1  
    return n * factorial(n - 1)
```

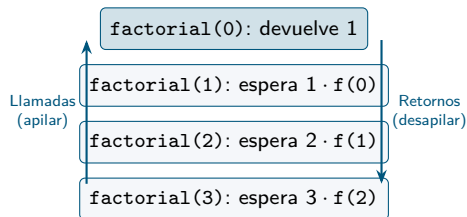
Lectura

Para calcular `factorial(n)`, primero necesito calcular `factorial(n - 1)`.

Ejemplo con $n = 4$:

$$\begin{aligned}4! &= 4 \cdot 3! \\ &= 4 \cdot 3 \cdot 2! \\ &= 4 \cdot 3 \cdot 2 \cdot 1! \\ &= 4 \cdot 3 \cdot 2 \cdot 1 \cdot 0! \\ &= 24.\end{aligned}$$

Pila de llamadas y memoria



Flujo de retornos: $1 \rightarrow 1 \rightarrow 2 \rightarrow 6$.

Marcos de activación:

Cada llamada suspende su ejecución y reserva memoria para parámetros, variables y dirección de retorno.

Stack Overflow: Si no hay caso base o n es muy grande, se agota la pila (RecursionError en Python).

Recursión e iteración

Una solución recursiva puede ser más clara, pero no necesariamente más eficiente.

Recursión	Iteración
Expresa naturalmente problemas definidos en términos de sí mismos.	Suele evitar el costo extra de las llamadas a función.
Es útil en árboles, búsqueda, divide y vencerás y definiciones inductivas.	Suele ser más directa cuando el problema es una repetición lineal.
Puede aumentar complejidad si se usa sin cuidado.	Puede requerir variables auxiliares que oculten la estructura matemática.

Fibonacci es un ejemplo clásico: la versión recursiva ingenua es clara, pero repite muchos cálculos.

Terminación y corrección

La recursión se conecta naturalmente con las ideas de corrección parcial y total.

Para una función recursiva queremos justificar dos cosas:

- **Corrección parcial:** si la función termina, entonces devuelve el resultado especificado.
- **Corrección total:** toda cadena de llamadas recursivas llega al caso base en un número finito de pasos.

$\text{corrección total} = \text{corrección parcial} + \text{terminación}.$

De código a recurrencias

```
def suma(n):  
    if n == 0:  
        return 0  
    return n + suma(n - 1)
```

Conteo en el modelo RAM:

- **Caso base ($n = 0$):**
Evaluar condición y retornar 0: $T(0) = c_1$.
- **Caso recursivo ($n > 0$):**
Evaluar, restar, sumar y retornar (c_2 pasos),
más la llamada recursiva:

$$T(n) = T(n - 1) + c_2.$$

Ecuación en recurrencia:

$$T(n) = \begin{cases} c_1, & n = 0, \\ T(n - 1) + c_2, & n > 0. \end{cases}$$

Desenrollado (expansión):

$$\begin{aligned} T(n) &= T(n - 1) + c_2 \\ &= T(n - 2) + 2c_2 \\ &\vdots \\ &= T(0) + n \cdot c_2 \\ &= c_1 + c_2 n \in \Theta(n). \end{aligned}$$

La recurrencia formaliza el costo ante la autorreferencia.

Divide y vencerás

Divide y vencerás es una estrategia de diseño algorítmico que resuelve un problema descomponiéndolo recursivamente:

- ➊ **Dividir:** separar el problema en subproblemas más pequeños del mismo tipo.
- ➋ **Conquistar:** resolver los subproblemas de forma recursiva (o directa si son pequeños).
- ➌ **Combinar:** unir las soluciones parciales para formar la solución global.

Forma común de la recurrencia

$$T(n) = a T\left(\frac{n}{b}\right) + f(n), \quad T(1) = \Theta(1).$$

- a : número de subproblemas en cada división ($a \geq 1$).
- b : factor de reducción de tamaño ($b > 1$).
- $f(n)$: costo no recursivo de **dividir** y **combinar**.

Ejemplo: potenciación entera

Problema

Dados $x \in \mathbb{R}$ y un entero $n \geq 0$, calcular la potencia x^n .

Enfoque iterativo ingenuo:

$$x^n = \underbrace{x \cdot x \cdot \dots \cdot x}_{n \text{ veces}}$$

Requiere $n - 1$ multiplicaciones:

$$T(n) \in \Theta(n).$$

Idea de Divide y Vencerás: Aprovechar propiedades algebraicas:

$$x^n = \begin{cases} 1, & n = 0, \\ \left(x^{n/2}\right)^2, & n \text{ par}, \\ x \cdot \left(x^{(n-1)/2}\right)^2, & n \text{ impar}. \end{cases}$$

Clave: Para calcular x^n , ¡basta con calcular $x^{\lfloor n/2 \rfloor}$ **una sola vez!**

Potenciación rápida: código y costo

```
def potencia(x, n):  
    if n == 0:  
        return 1  
    mitad = potencia(x, n // 2)  
    if n % 2 == 0:  
        return mitad * mitad  
    else:  
        return x * mitad * mitad
```

Al guardar *mitad*, realizamos **una sola** llamada recursiva con $n/2$.

Estructura Divide y Vencerás:

- **Dividir:** calcular $n//2$ en tiempo $\Theta(1)$.
- **Conquistar:** $a = 1$ subproblema de tamaño $n/2$ ($b = 2$).
- **Combinar:** a lo sumo 2 multiplicaciones en tiempo $\Theta(1)$.

Recurrencia resultante:

$$T(n) = T\left(\frac{n}{2}\right) + \Theta(1)$$

$$\implies T(n) \in \Theta(\log n).$$

Algunos métodos para resolver recurrencias son:

- **Sustitución:** se propone una cota y se demuestra por inducción matemática.
- **Árbol recursivo:** se representa el costo por niveles y se suman los costos de todos los niveles.
- **Método maestro:** se aplica a recurrencias de la forma $T(n) = aT(n/b) + f(n)$.
- **Akra-Bazzi:** generaliza el método maestro para recurrencias más flexibles.

En esta primera aproximación desarrollaremos sustitución y árbol recursivo.

Método de sustitución

El método de sustitución tiene dos pasos:

- 1 **Proponer** la forma de la solución.
- 2 **Probar** la cota por inducción matemática.

La dificultad principal es elegir una buena conjetura.

Idea

Si sospechamos que $T(n) \in \Theta(g(n))$, intentamos demostrar:

$$c_1g(n) \leq T(n) \leq c_2g(n)$$

para constantes positivas c_1, c_2 y para n suficientemente grande.

Sustitución: ejemplo 1

Considere:

$$T(1) = d, \quad T(n) = T(n-1) + cn, \quad c, d > 0.$$

Conjeturamos que $T(n) \leq an^2$ para alguna constante $a > 0$.

- **Paso inductivo:** Suponiendo $T(n-1) \leq a(n-1)^2$:

$$T(n) \leq a(n-1)^2 + cn = an^2 - (2a - c)n + a.$$

Para que esto sea $\leq an^2$, requerimos $-(2a - c)n + a \leq 0$. Si elegimos $a \geq c$, entonces para todo $n \geq 1$:

$$-(2a - c)n + a \leq -an + a = -a(n-1) \leq 0.$$

- **Caso base ($n = 1$):**

$$T(1) = d \leq a(1)^2 \iff a \geq d.$$

Conclusión: Fijando $a \geq \max(c, d)$, la cota se cumple para todo $n \geq 1$, por lo que $T(n) \in \mathcal{O}(n^2)$. Con una cota inferior análoga, $T(n) \in \Theta(n^2)$.

Sustitución: ejemplo 2

Considere:

$$T(1) = d, \quad T(n) = T\left(\frac{n}{2}\right) + cn, \quad c, d > 0.$$

Conjeturamos que $T(n) \leq an$ para alguna constante $a > 0$.

- **Paso inductivo:** Suponiendo $T(n/2) \leq a(n/2)$:

$$T(n) \leq a\frac{n}{2} + cn = \left(\frac{a}{2} + c\right)n.$$

Para que sea $\leq an$, requerimos $\frac{a}{2} + c \leq a \iff a \geq 2c$.

- **Caso base ($n = 1$):**

$$T(1) = d \leq a(1) \iff a \geq d.$$

Conclusión: Eligiendo $a \geq \max(2c, d)$, se cumple $T(n) \leq an$ para todo $n \geq 1$. Como además $T(n) \geq cn$, concluimos que $T(n) \in \Theta(n)$.

Método del árbol recursivo

En el método del árbol recursivo:

- Cada nodo representa un subproblema.
- La etiqueta del nodo representa el costo no recursivo de ese subproblema.
- Cada nivel agrupa subproblemas con el mismo número de divisiones.
- El costo total se obtiene sumando los costos por nivel.

$$\text{costo total} = \sum_{\text{niveles}} \text{costo del nivel} + \text{costo de las hojas}.$$

Este método ayuda a construir una conjetura antes de hacer una prueba formal por sustitución.

Árbol recursivo: ejemplo 1

Considere:

$$T(n) = T(n-1) + cn, \quad T(1) = d.$$

El árbol tiene una sola rama:

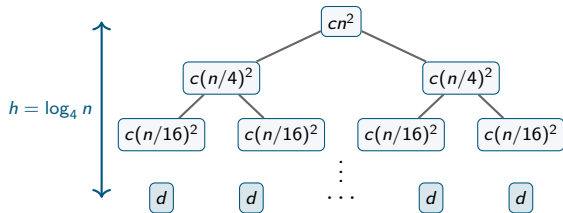
$$\begin{aligned} T(n) &= cn + c(n-1) + c(n-2) + \cdots + c(2) + d \\ &= c \sum_{k=2}^n k + d \\ &= c \left(\frac{n(n+1)}{2} - 1 \right) + d. \end{aligned}$$

Por lo tanto:

$$T(n) \in \Theta(n^2).$$

Árbol recursivo: ejemplo 2

Considere $T(n) = 2T(n/4) + cn^2$ con $T(1) = d$.



$$= cn^2$$

$$= 2 \cdot c(n/4)^2 = \frac{1}{8} cn^2$$

$$= 4 \cdot c(n/16)^2 = \frac{1}{64} cn^2$$

$$= 2^{\log_4 n} \cdot d = d\sqrt{n}$$

Árbol recursivo: ejemplo 2 (cálculo)

La altura del árbol es $h = \log_4 n$.

Suma de costos internos (serie geométrica decreciente):

$$\sum_{i=0}^{\log_4 n - 1} cn^2 \left(\frac{1}{8}\right)^i \leq cn^2 \sum_{i=0}^{\infty} \left(\frac{1}{8}\right)^i = cn^2 \left(\frac{1}{1 - 1/8}\right) = \frac{8}{7}cn^2.$$

Costo de las hojas en la base:

$$2^{\log_4 n} \cdot d = 4^{(\log_4 n)/2} \cdot d = d\sqrt{n}.$$

Costo total:

$$T(n) \leq \frac{8}{7}cn^2 + d\sqrt{n} \implies T(n) \in \Theta(n^2).$$

El término dominante proviene de la **raíz** (cn^2); las llamadas inferiores aportan cada vez menos.